A CASE STUDY REPORT ON

Cloud Identity, Authentication and Authorization

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN PARTIAL FULFILMENT OF THE REQUIREMENTS OF THE SUBJECT

CLOUD COMPUTING
CASE STUDY

THIRD YEAR AI & DS ENGINEERING

SUBMITTED BY

1. Sujit Bera (TE-07)
2. Onkar Kharat (TE-39)
3. Ritesh Gaike (TE-21)
4. Omkar Shedage (TE-56)
5. Tanmay Virkar (TE-65)
6. Himanshu Jadhav (TE-32)



DEPARTMENT OF AI & DS ENGINEERING
JSPM NARHE TECHNICAL CAMPUS, NARHE, PUNE



JAYAWANT SHIKSHAN PRASARAK MANDAL

Group of Institutes

DEPARTMENT OF AI & DS ENGINEERING
JSPM NARHE TECHNICAL CAMPUS, PUNE

CERTIFICATE

This is to certify that the following students of Third Year AI & DS Engineering:

1. Sujit Bera (TE-07)
2. Onkar Kharat (TE-39)
3. Ritesh Gaike (TE-21)
4. Omkar Shedage (TE-56)
5. Tanmay Virkar (TE-65)
6. Himanshu Jadhav (TE-32)

have successfully completed the term work for the subject “Cloud Computing Case Study” as prescribed by Savitribai Phule Pune University, Pune, during the academic year 2026–27.

Project Title: Cloud Identity, Authentication and Authorization

Prof. Snehal L. Khot
Subject Teacher

Dr. Manisha Hajare
Head of Department (HOD)

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our subject teacher, Prof. Snehal L. Khot, for valuable guidance, encouragement, and constructive feedback throughout the preparation of this case study report.

We are thankful to the Head of the Department and the Principal of JSPM Narhe Technical Campus for providing the facilities and academic environment required for this work. We also acknowledge the researchers and reliable resources consulted during the preparation of this report.

Finally, we thank our teammates and everyone who supported us in completing this case study.

Submitted by:

Sujit Bera (TE-07)
Onkar Kharat (TE-39)
Ritesh Gaike (TE-21)
Omkar Shedage (TE-56)
Tanmay Virkar (TE-65)
Himanshu Jadhav (TE-32)

ABSTRACT

As organisations move applications and data to public clouds, the network perimeter has lost its role as the main security boundary and identity has taken its place. Every user, device, application and pipeline that touches a cloud resource must be identified, authenticated and then authorized to perform only the actions it needs. This case study examines the concepts, standards and architectures that make this possible, and relates them to the confidentiality, integrity and availability goals of cloud security.

The report defines cloud identity and the AAA model, compares SAML 2.0, OAuth 2.0, OpenID Connect and Kerberos, evaluates authentication mechanisms from passwords to passkeys, compares ACL, RBAC and ABAC, explains default-deny policy evaluation, discusses authorization challenges in the cloud, and maps the identity services of AWS, Azure and Google Cloud. It lists requirements and tests for secure cloud software and proposes a layered identity-centric reference architecture. Two small programs, a JWT validator and a default-deny policy evaluator, were run on test cases; they verify the logic of simplified code and are not measurements of real services. The study concludes that strong authentication, federation, short-lived credentials, least privilege and continuous monitoring, guided by Zero Trust, substantially reduce identity-related risk.

Keywords: *cloud IAM, authentication, authorization, SSO, SAML, OAuth 2.0, OpenID Connect, JWT, RBAC, ABAC, Zero Trust, secure cloud software.*

TABLE OF CONTENTS

1. INTRODUCTION 6
1.1 Background 6
1.2 Importance and Relevance 6
1.3 Scope and Method 6
2. PROBLEM STATEMENT AND OBJECTIVES 7
3. CASE STUDY AND BACKGROUND ANALYSIS 7
4. PROPOSED SOLUTION AND METHODOLOGY 8
5. TECHNOLOGIES AND TECHNIQUES USED 9
5.1 Authentication mechanisms 9
5.2 SAML 2.0, OAuth 2.0, OpenID Connect and JWT 9
5.3 Authorization models and policy evaluation 10
5.4 Workload identity, privileged access and Zero Trust 11
5.5 Authorization challenges in the cloud 11
6. SECURE CLOUD SOFTWARE: REQUIREMENTS AND TESTING 11
7. IMPLEMENTATION AND RESULTS 12
7.1 Workflow, policy and validation logic 12
7.2 Test results 12
8. ADVANTAGES AND LIMITATIONS 13
9. APPLICATIONS AND FUTURE SCOPE 14
10. CONCLUSION 14
11. REFERENCES 14

1. INTRODUCTION

1.1 Background

In a conventional data centre a firewall marked the trust boundary: traffic inside was largely trusted and traffic outside was not. In the cloud, resources are created and managed through public APIs and web consoles and can be reached from anywhere [1]. The network perimeter therefore weakens as a control, and **identity becomes the primary control plane** of cloud security.

Cloud identity is the digital representation of every entity that can act in a cloud environment: employees, partners, customers, devices, applications, containers and pipelines. **Authentication** (AuthN) verifies that an entity is who it claims to be; **authorization** (AuthZ) decides what the verified entity may do; and **accounting** records what it did. Together these form the AAA model that underpins Identity and Access Management (IAM) (Figure 1).

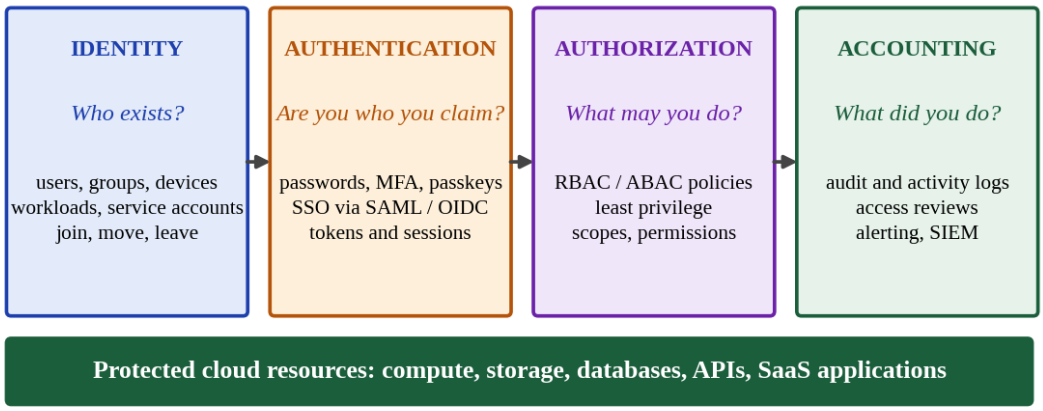


Figure 1: Pillars of cloud Identity and Access Management (IAM)

1.2 Importance and Relevance

Cloud providers operate under a **shared responsibility model**: the provider secures the underlying infrastructure, while the customer remains responsible for identities, permissions and data protection [2], [3], [4]. Weak authentication, over-permissive roles and unmanaged credentials are therefore customer-side risks. The Cloud Security Alliance treats IAM as a dedicated security domain [5]. IAM is also how the classic security goals of Unit IV are enforced in practice, as Table 1 shows, and it supplies the evidence that regulation expects, such as India's Digital Personal Data Protection Act, 2023 [6].

Table 1: How identity controls support confidentiality, integrity and availability

Property	Identity controls that support it	Typical failure
Confidentiality	Least-privilege authorization, MFA, access to encryption keys gated by IAM	Stolen password gives access to private data
Integrity	Write permissions limited by role; signed tokens and assertions; audit logs; separation of duties	Over-broad role lets a user alter or delete records
Availability	Resilient identity provider, break-glass accounts, rate limiting, short-lived credentials renewed automatically	IdP outage or lockout stops all access

1.3 Scope and Method

This is a literature and architecture-based case study of concepts, standards and provider documentation, covering SAML 2.0, OAuth 2.0, OpenID Connect, JSON Web Tokens,

authorization models, workload identity, Zero Trust and the identity services of AWS, Azure and Google Cloud. Product configuration and penetration testing are out of scope. In Section 7, two small programs written for this report were run locally: a token validator and a default-deny policy evaluator. They are unit tests of simplified code, not measurements of real cloud services.

2. PROBLEM STATEMENT AND OBJECTIVES

Enterprises combine on-premises directories, several public clouds and dozens of SaaS applications. Each brings its own accounts, roles and policy language, and the fragmentation causes recurring problems: credential theft through phishing and password reuse; over-privileged roles granted to avoid breaking applications; long-lived access keys embedded in code; orphaned accounts of departed staff; weakly governed machine identities; and poor visibility into who accessed what. Risk management for identity follows the usual cycle of identifying assets and threats, assessing likelihood and impact, applying controls and monitoring residual risk, and should be part of enterprise-wide risk management rather than a separate technical task [7]. Table 2 lists the main identity-related risks.

Table 2: Identity-related cloud risks and their controls

Risk	Example	CIA	Primary controls
Credential theft	Phishing, credential stuffing	C	Phishing-resistant MFA, passkeys, breached-password checks
Over-privileged roles	Admin role given to a developer	C, I	Least privilege, permission boundaries, access reviews
Long-lived secrets	Access key committed to a repository	C	Short-lived tokens, workload federation, secrets manager
Misconfigured trust	Wrong account can assume a role	C, I	Strict audience and subject conditions, policy review
Token misuse	Stolen bearer token replayed	C, I	Short expiry, full validation, secure storage
IdP compromise or outage	Central identity service fails	A, C	High availability, monitoring, break-glass accounts
Insider misuse	Authorised user abuses access	C, I	Separation of duties, PAM, audit alerts

Objectives. (i) Explain identity, authentication and authorization in a cloud context; (ii) compare SAML 2.0, OAuth 2.0, OpenID Connect and Kerberos; (iii) evaluate authentication mechanisms and compare ACL, RBAC and ABAC; (iv) explain cloud policy evaluation and the authorization challenges of the cloud; (v) map the identity services of AWS, Azure and Google Cloud; (vi) state requirements and tests for secure cloud software; and (vii) propose and test a layered reference architecture.

3. CASE STUDY AND BACKGROUND ANALYSIS

Early enterprise systems used directory services and ticket-based protocols: LDAP stored accounts centrally and Kerberos let users on a trusted network obtain tickets without resending passwords [8]. Web applications then needed sign-in across organisations, which led to SAML 2.0 [9]; mobile apps and APIs needed delegated access without sharing passwords, which produced OAuth 2.0 [10] and OpenID Connect on top of it [11]. Cloud platforms added fine-grained policy-based IAM, and Zero Trust now generalises the idea that no request is trusted

merely because of its network location [12]. Table 3 compares the protocols and Table 4 maps the three platforms.

Table 3: Comparison of identity and access protocols

Protocol	Purpose	Format	Typical use
SAML 2.0	Federated web SSO (authentication)	Signed XML assertion	Enterprise SSO to SaaS and cloud consoles
OAuth 2.0	Delegated authorization to APIs	Opaque or JWT access token	API access, mobile and single-page apps
OpenID Connect	Authentication layer on OAuth 2.0	JWT ID token	Modern web and mobile login
Kerberos	Mutual authentication in a trusted realm	Encrypted tickets	On-premises Windows / Active Directory

Table 4: Identity concepts mapped to cloud provider services

Concept	AWS	Microsoft Azure	Google Cloud
Workforce IdP / SSO	IAM Identity Center	Microsoft Entra ID	Cloud Identity, Workforce Identity Federation
Customer identity	Amazon Cognito	Entra External ID	Identity Platform
Authorization	IAM policies and roles	Azure RBAC	IAM roles and bindings
Workload identity	IAM roles for services	Managed identities	Service accounts
Secrets	Secrets Manager	Key Vault	Secret Manager
Audit logs	CloudTrail	Activity and Entra logs	Cloud Audit Logs

Key principles used below: **least privilege** (only the permissions needed, for only as long as needed); **default deny** (access refused unless explicitly allowed); **separation of duties**; **defence in depth**; and **short-lived credentials**, which reduce the value of any stolen secret.

4. PROPOSED SOLUTION AND METHODOLOGY

The proposed approach is an **identity-centric, layered architecture** in which the identity provider is the single source of truth for who a principal is and each cloud platform is the enforcement point for what that principal may do. It has six stages: (1) establish authoritative identities with automated joiner-mover-leaver provisioning (SCIM); (2) strengthen authentication with MFA and passkeys; (3) federate access to clouds and applications through SAML 2.0 or OIDC; (4) authorize with least privilege using RBAC plus ABAC conditions; (5) secure workload and privileged access with short-lived tokens, a secrets manager and just-in-time elevation; and (6) monitor, audit and recertify access continuously. Figure 2 shows the architecture, which is conceptual and not tied to a commercial product.

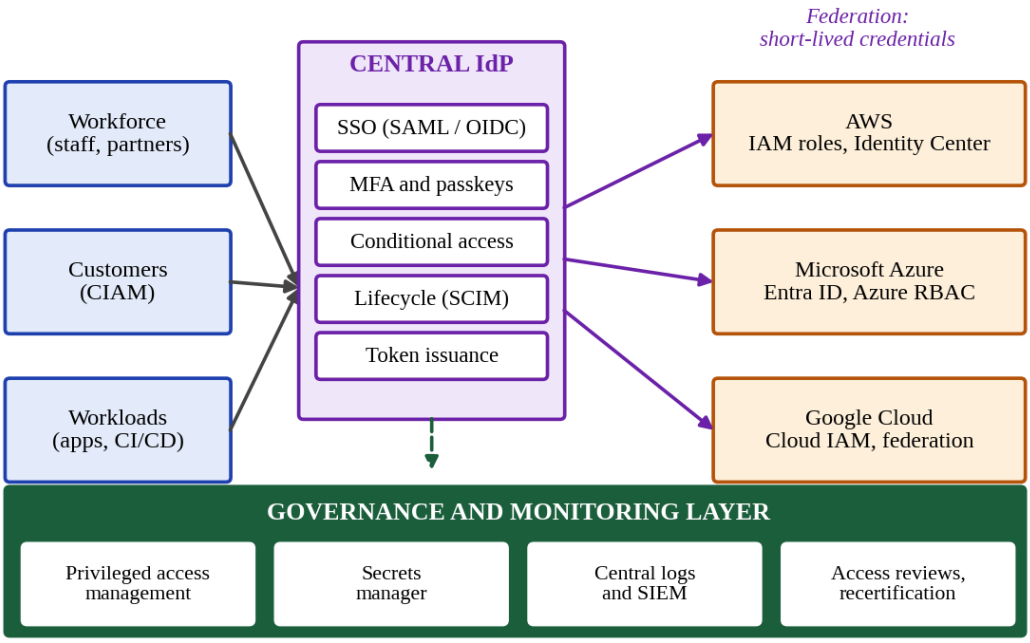


Figure 2: Reference architecture for federated, least-privilege cloud access

5. TECHNOLOGIES AND TECHNIQUES USED

5.1 Authentication mechanisms

Mechanisms are classified by factor: something known, something possessed and something inherent. NIST SP 800-63-4 recommends allowing long passwords, screening them against lists of compromised values, and avoiding composition rules and periodic rotation without evidence of compromise [13], [14]. Table 5 compares common mechanisms.

Table 5: Common authentication mechanisms

Mechanism	Factor	Strength	Limitation
Password only	Knowledge	Universal	Phishable, reusable, guessable
TOTP app code	Possession	Offline, inexpensive	Can be relayed by phishing sites
Push notification	Possession	Convenient	Prompt-fatigue attacks
FIDO2 / WebAuthn passkey	Possession + PIN or biometric	Phishing-resistant, origin-bound keys	Needs compatible devices; recovery planning
Client certificate (mTLS)	Possession	Strong for machines	Certificate lifecycle overhead

FIDO2 and WebAuthn use public-key cryptography: the private key never leaves the authenticator and each credential is bound to the website's origin, which defeats look-alike phishing pages [15]. Passkeys are therefore preferred for administrators and other high-risk accounts.

5.2 SAML 2.0, OAuth 2.0, OpenID Connect and JWT

SAML 2.0. The identity provider passes a digitally signed XML assertion to the service provider [9]. In the SP-initiated flow the user requests an application, is redirected to the IdP, signs in (with MFA), and the browser carries the assertion back to the service provider's Assertion Consumer Service, which must validate the signature, issuer, audience and validity

period before creating a session. SAML is mature and widely supported by enterprise SaaS, but its XML is verbose and it suits native mobile apps poorly.

OAuth 2.0 is a framework for *delegated authorization*, not authentication [10]. The recommended Authorization Code flow with PKCE (Figure 3) has the client send only a hash of a random secret (the code challenge) in the authorization request and later prove possession of the secret (the code verifier) when exchanging the code for tokens, so a stolen code is useless to an attacker [16]. Current guidance advises against the implicit and password grants and recommends exact redirect-URI matching [17]. A bearer token works for whoever holds it, so token theft is the main residual risk.

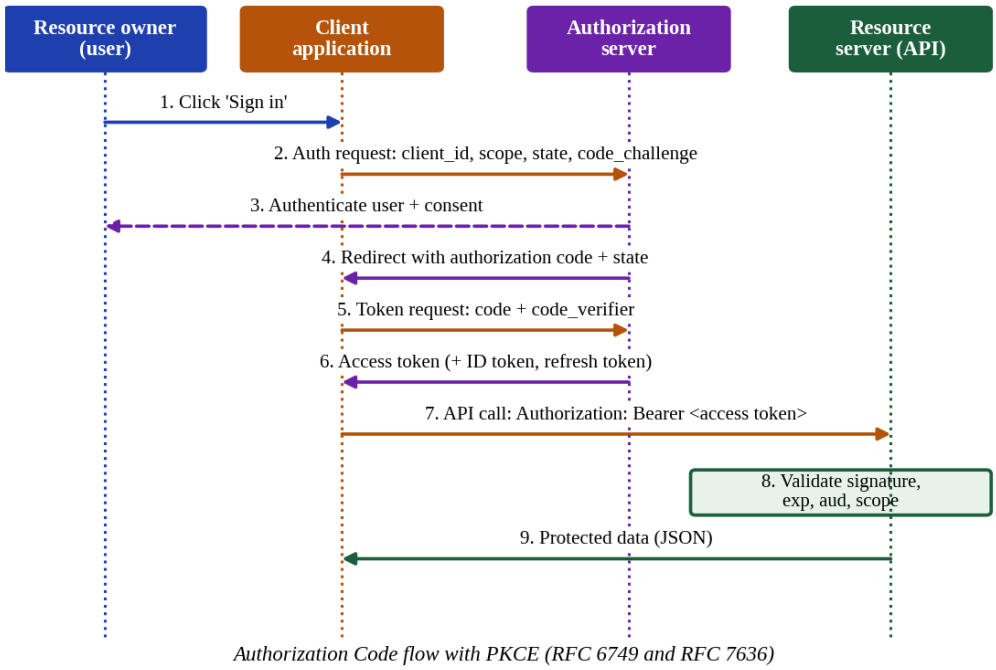


Figure 3: OAuth 2.0 Authorization Code flow with PKCE

OpenID Connect adds an identity layer: the authorization server also returns an *ID token* stating who authenticated, when and for which client, and publishes a discovery document at /.well-known/openid-configuration [11]. Because it uses JSON and REST, OIDC suits mobile apps and cloud-native services better than SAML and underlies workload identity federation in all three major clouds. A **JSON Web Token** has three Base64url parts (header, payload, signature) [18]. The payload carries claims such as issuer (iss), subject (sub), audience (aud), expiry (exp) and scope, and the signature lets any holder of the issuer's public key detect tampering. A JWT is signed, not encrypted by default, so sensitive data must not be placed in it, and the verifier must fix the allowed algorithms and never accept "none" [19].

5.3 Authorization models and policy evaluation

RBAC assigns permissions to roles and roles to users, which simplifies administration and auditing [20]. **ABAC** evaluates attributes of the subject, resource, action and environment against rules, allowing fine-grained, context-aware decisions [21]. An **ACL** attaches permissions directly to an object. Cloud platforms usually combine RBAC with ABAC-style conditions such as "the session used MFA" or "the resource has tag X" (Table 6).

Table 6: Comparison of authorization models

Aspect	ACL	RBAC	ABAC
Decision basis	Per-object entries	User's role	Attributes plus policy rules
Granularity	Object	Role	Very fine
Administration	Hard at scale	Easy to moderate	Complex policies
Context awareness	None	Low	High (time, device, location)
Main risk	Inconsistent entries	Role explosion	Policy errors, hard to test

Cloud platforms apply **default deny**: in AWS IAM a request is denied unless an applicable policy explicitly allows it, and any explicit deny overrides every allow; organisation-level guardrails such as Service Control Policies and permission boundaries cap the maximum permissions that identity policies can grant [2]. Azure uses deny assignments and Google Cloud uses deny policies, with details that differ by provider [3], [4]. Figure 4 shows the simplified decision sequence.

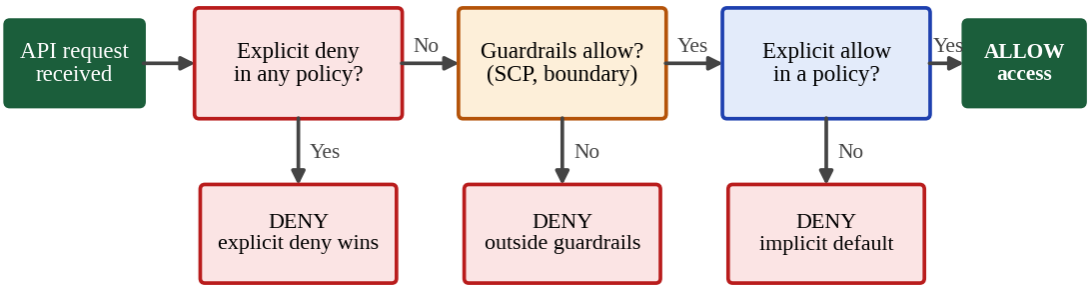


Figure 4: Simplified policy evaluation with default deny

5.4 Workload identity, privileged access and Zero Trust

Applications should authenticate as *workloads*, not with long-lived keys: IAM roles, managed identities and service accounts supply temporary credentials automatically. For code outside the cloud, such as CI/CD pipelines, workload identity federation exchanges the pipeline's OIDC token for short-lived cloud credentials, removing stored secrets. Remaining secrets belong in a secrets manager with rotation and access logging, and privileged access management grants administrator roles just-in-time, with approval and session recording. NIST SP 800-207 defines Zero Trust as removing implicit trust based on network location and evaluating every request using identity, device state and other context through a policy engine, policy administrator and policy enforcement point [12]; IAM is the core of that design.

5.5 Authorization challenges in the cloud

- **Multi-tenancy and shared responsibility:** customers must configure isolation and permissions correctly; a single mistake can expose a whole account.
- **Policy complexity and fragmentation:** each provider has its own policy language and semantics, so uniform multi-cloud policy is hard to achieve and test.
- **Permission creep and role explosion:** roles accumulate over time unless access is reviewed regularly.
- **Delegation (confused deputy):** a service acting for a user may use its own broader permissions unless scopes and audiences are enforced.

- **Revocation delay:** a signed JWT stays valid until it expires, and policy changes can take time to propagate, so short lifetimes and continuous evaluation matter.
- **Machine identities:** workloads outnumber people and receive less governance.

6. SECURE CLOUD SOFTWARE: REQUIREMENTS AND TESTING

Applications built on cloud IAM must themselves meet security requirements [22], [23]. Core requirements are: authenticate every request and enforce authorization on the server side; validate tokens fully (signature, algorithm, issuer, audience, expiry, scope); apply default deny and least privilege to each service; keep secrets out of code and configuration; require MFA for administrators; and log security events without recording secrets or tokens. Table 7 lists the matching tests, which should run in the CI/CD pipeline and again after each change.

Table 7: Security tests for identity-aware cloud software

Test	What it checks	Example
Secret scanning, static analysis	Hard-coded keys, insecure token handling	Scan the repository for access keys on every commit
Token validation unit tests	Rejection of expired, wrong-audience, tampered and unsigned tokens	Section 7 test cases
Policy simulation and analysis	Effective permissions and unintended access	Provider policy simulators and access analysers
Negative integration tests	Default deny; cross-tenant access blocked	A request without MFA must fail
Configuration scanning	Public storage, missing MFA, stale roles	Scheduled scan of all accounts
Authorised penetration test	Redirect-URI abuse, session weaknesses	Scoped test with written approval

7. IMPLEMENTATION AND RESULTS

7.1 Workflow, policy and validation logic

In the proposed architecture a joiner is created in the HR system and provisioned to the IdP; the user signs in with a passkey or password plus MFA; the IdP applies conditional-access rules and issues a SAML assertion or OIDC tokens; the cloud platform maps the federated identity to a role and issues short-lived credentials; every API call is evaluated against policy and denied by default; all events go to logs and a SIEM; and deprovisioning removes access everywhere when the user leaves. The policy below is an illustrative example (hypothetical resource names) that allows read-only access to one bucket only for MFA sessions.

```
{ "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::example-reports", "arn:aws:s3:::example-reports/*"],
    "Condition": {"Bool": {"aws:MultiFactorAuthPresent": "true"}}
  }] }
```

Listing 1: Illustrative least-privilege IAM policy (AWS JSON syntax)

```
def validate(token):
    header, payload, sig = decode(token)
    if header["alg"] not in ALLOWED ALGS: reject("algorithm")   # never "none"
    verify signature(JWKS[header["kid"]], token)                # issuer public key
    if payload["iss"] != EXPECTED_ISSUER: reject("issuer")
    if EXPECTED_AUDIENCE not in aud(payload): reject("audience")
    if now() >= payload["exp"]: reject("expired")
    if REQUIRED SCOPE not in scope(payload): reject("scope")
    return payload["sub"]
```

Listing 2: Token validation logic (condensed from the program that was run)

7.2 Test results

The validator (RS256, 2048-bit RSA key) was run against eight tokens, and the default-deny evaluator against six requests modelled on Listing 1. All outcomes matched the expected result (Table 8 and Table 9). These are unit tests of simplified code written by the authors, so they show that the logic behaves as designed; they do not show that any real provider or application is secure.

Table 8: Token validation tests (executed)

Test case	Outcome	Reason	Expected result met
Valid token	Accept	ok	Yes
Expired token	Reject	expired	Yes
Wrong audience	Reject	wrong audience	Yes
Wrong issuer	Reject	wrong issuer	Yes
Missing scope	Reject	missing scope	Yes
alg = none	Reject	algorithm not allowed	Yes
Signed by another key	Reject	bad signature	Yes
Tampered payload	Reject	bad signature	Yes

Table 9: Default-deny policy evaluation (executed)

Request	Decision	Reason	Matches design
Analyst reads report, MFA	ALLOW	explicit allow	Yes
Analyst reads report, no MFA	DENY	implicit deny	Yes
Developer deletes production DB	DENY	explicit deny	Yes
Pipeline deploys to test account	ALLOW	explicit allow	Yes
Pipeline deploys to production	DENY	implicit deny	Yes
Request to unlisted service	DENY	outside guardrail	Yes

Observations. (i) Full validation matters: tampered and unsigned tokens fail only because the signature and algorithm checks come before any claim is trusted. (ii) The “no MFA” and “production deploy” cases are denied simply because no allow matches, which shows why default deny is safer than trying to list everything forbidden. (iii) Guardrails and explicit denies override allows, so one organisation-level rule can contain a mistake in a team-level policy. (iv) Security is only as strong as the weakest link, so token validation, redirect-URI checks and the security of the IdP itself deserve the same care as the policies.

8. ADVANTAGES AND LIMITATIONS

Table 10: Advantages and limitations of the identity-centric approach

Advantages	Limitations
Central control and consistent policy across clouds	Central IdP is a high-value target and single point of failure
Better user experience through SSO and passkeys	Migration and integration effort can be significant
Reduced credential risk with short-lived tokens	ABAC and conditional policies are harder to test and audit
Strong audit trail supports compliance	Provider differences hinder true multi-cloud uniformity
Aligns with Zero Trust principles	Needs ongoing governance; stale roles accumulate

Stronger authentication adds friction, so a risk-based approach (smooth for low-risk requests, step-up for high-risk ones) balances security and usability. Organisations must also plan account recovery and break-glass procedures so that strict controls do not lock out legitimate administrators.

9. APPLICATIONS AND FUTURE SCOPE

- **Applications:** enterprise workforce SSO and MFA; customer identity (CIAM) for consumer applications; multi-cloud governance; keyless CI/CD pipelines through OIDC federation; regulated sectors such as banking, healthcare and education.
- **Future scope:** wider passkey adoption; continuous access evaluation that re-assesses sessions when risk changes; entitlement management tools that find unused permissions; decentralised identity and verifiable credentials; governance of AI-agent and other non-human identities; and migration of signatures to post-quantum algorithms.

10. CONCLUSION

Identity has replaced the network perimeter as the main security boundary of the cloud, and it is the mechanism through which confidentiality, integrity and availability are enforced. Effective protection combines strong, preferably phishing-resistant authentication; standards-based federation through SAML 2.0 and OpenID Connect; delegated authorization through OAuth 2.0 with PKCE; and least-privilege policies evaluated on a default-deny basis, supported by short-lived credentials, secure software practices with matching tests, and continuous monitoring. The executed tests confirm that full token validation and default deny behave as designed in simplified code. The main limits are the concentration of risk in the identity provider and the ongoing governance effort required.

11. REFERENCES

[1] P. Mell and T. Grance, “The NIST definition of cloud computing,” NIST Special Publication 800-145, Sep. 2011.

[2] Amazon Web Services, “AWS Identity and Access Management User Guide.” [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/

[3] Microsoft, “Microsoft Entra ID documentation.” [Online]. Available: https://learn.microsoft.com/entra/identity/

[4] Google Cloud, “Identity and Access Management (IAM) documentation.” [Online]. Available: https://cloud.google.com/iam/docs

[5] Cloud Security Alliance, “Security Guidance for Critical Areas of Focus in Cloud Computing v4.0,” 2017. [Online]. Available: https://cloudsecurityalliance.org/research/guidance

[6] Government of India, “The Digital Personal Data Protection Act, 2023,” Act No. 22 of 2023.

[7] K. Hashizume, D. G. Rosado, E. Fernández-Medina, and E. B. Fernandez, “An analysis of security issues for cloud computing,” J. Internet Serv. Appl., vol. 4, art. 5, 2013.

[8] C. Neuman, T. Yu, S. Hartman, and K. Raeburn, “The Kerberos Network Authentication Service (V5),” IETF RFC 4120, Jul. 2005.

[9] OASIS, “Assertions and Protocols for the OASIS Security Assertion Markup Language (SAML) V2.0,” OASIS Standard, Mar. 2005.

[10] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” IETF RFC 6749, Oct. 2012.

[11] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID Connect Core 1.0 incorporating errata set 1,” OpenID Foundation, Nov. 2014.

[12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST Special Publication 800-207, Aug. 2020. doi: 10.6028/NIST.SP.800-207.

[13] D. Temoshok et al., “Digital Identity Guidelines,” NIST Special Publication 800-63-4, Jul. 2025. doi: 10.6028/NIST.SP.800-63-4.

[14] OWASP Foundation, “Authentication Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[15] W3C, “Web Authentication: An API for accessing Public Key Credentials, Level 2,” W3C Recommendation, Apr. 2021.

[16] N. Sakimura, J. Bradley, and N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients,” IETF RFC 7636, Sep. 2015.

[17] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, “Best Current Practice for OAuth 2.0 Security,” IETF RFC 9700, Jan. 2025.

[18] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.

[19] Y. Sheffer, D. Hardt, and M. Jones, “JSON Web Token Best Current Practices,” IETF RFC 8725, Feb. 2020.

[20] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” ACM Trans. Inf. Syst. Secur., vol. 4, no. 3, pp. 224–274, Aug. 2001.

[21] V. C. Hu et al., “Guide to Attribute Based Access Control (ABAC) Definition and Considerations,” NIST Special Publication 800-162, Jan. 2014.

[22] OWASP Foundation, “OWASP Application Security Verification Standard (ASVS).” [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[23] M. Souppaya, K. Scarfone, and D. Dodson, “Secure Software Development Framework (SSDF) Version 1.1,” NIST Special Publication 800-218, Feb. 2022.